

PYTHON FULL STACK WEB DEVELOPMENT- Task 2

Implement User Authentication System (Register, Login, Logout) using Flask

1. Objective

The objective of Task-2 is to help interns understand **how real web applications manage users securely**.

In this task, you will extend your Task-1 project by adding:

- User registration
- User login
- Session management
- Access control

This task introduces **backend security fundamentals**, which are mandatory for **any production-level web application**.

2. Industry Context (Why This Task Is Critical)

Almost every real application includes:

- Login & Signup
- Protected dashboards
- Session-based authentication

Python Full Stack Developers are expected to:

- Secure user data
- Restrict unauthorized access
- Manage user sessions correctly

This task reflects **real SaaS and enterprise application workflows**.

3. Real-World Problem Statement

“Only registered users should be able to access the dashboard and manage data.”

You must ensure:

- Users can create accounts
 - Only logged-in users can access protected pages
 - Users can log out safely
-

4. What You Will Build (Clearly Defined)

You will upgrade your existing application to include:

Features

1. User Registration (Signup)
 2. User Login
 3. Password Hashing (Security)
 4. Session-based Authentication
 5. Protected Dashboard Page
 6. Logout Functionality
-

5. Tech Stack

- Backend: Python + Flask
 - Frontend: HTML, CSS
 - Database: SQLite
 - Security: Werkzeug Password Hashing
 - Sessions: Flask built-in session
-

6. Updated Project Structure

cpp

```
python-fullstack-task2/  
|  
├─ app.py  
├─ database.db  
├─ templates/  
|   ├─ login.html  
|   ├─ register.html  
|   └─ dashboard.html  
└─ static/  
    └─ style.css
```

7. Database Design (Important)

New Table: `users`

pgsql

```
id (INTEGER, Primary Key)  
username (TEXT, UNIQUE)  
password (TEXT, Hashed)
```

8. Step-by-Step Implementation Guide

Step 1: Install Required Packages

bash

```
pip install flask werkzeug
```

Step 2: Backend Logic (Authentication)

app.py (Core Authentication Logic)

python

```
from flask import Flask, render_template, request, redirect, session, url_for
from werkzeug.security import generate_password_hash, check_password_hash
import sqlite3

app = Flask(__name__)
app.secret_key = "secure-secret-key"

def get_db():
    conn = sqlite3.connect("database.db")
    conn.row_factory = sqlite3.Row
    return conn

@app.route('/register', methods=['GET', 'POST'])
def register():
    if request.method == 'POST':
        username = request.form['username']
        password = generate_password_hash(request.form['password'])

        db = get_db()
        db.execute(
            "INSERT INTO users (username, password) VALUES (?, ?)",
            (username, password)
        )
        db.commit()
        return redirect('/login')

    return render_template('register.html')

@app.route('/login', methods=['GET', 'POST'])
def login():
    if request.method == 'POST':
        username = request.form['username']
        password = request.form['password']

        db = get_db()
        user = db.execute(
            "SELECT * FROM users WHERE username = ?", (username,)
        ).fetchone()

        if user and check_password_hash(user['password'], password):
            session['user'] = user['username']
            return redirect('/dashboard')

    return render_template('login.html')

@app.route('/dashboard')
def dashboard():
    if 'user' not in session:
        return redirect('/login')
    return render_template('dashboard.html', user=session['user'])

@app.route('/logout')
def logout():
    session.pop('user', None)
    return redirect('/login')

if __name__ == '__main__':
    app.run(debug=True)
```

Step 3: Frontend Templates

register.html

```
html

<form method="post">
  <h2>Register</h2>
  <input name="username" placeholder="Username" required>
  <input type="password" name="password" placeholder="Password" required>
  <button>Create Account</button>
</form>
```

login.html

```
html

<form method="post">
  <h2>Login</h2>
  <input name="username" placeholder="Username" required>
  <input type="password" name="password" placeholder="Password" required>
  <button>Login</button>
</form>
```

dashboard.html

```
html

<h2>Welcome, {{ user }}</h2>
<p>This is a protected dashboard.</p>
<a href="/logout">Logout</a>
```

9. How to Run the Project

```
bash

python app.py
```

Open browser:

```
arduino

http://127.0.0.1:5000/register
```

10. Security Concepts You Learn

- Password hashing (never store plain text passwords)
- Session-based authentication
- Protected routes
- Secure logout

These concepts are mandatory in interviews.

11. Deliverables

Each intern must submit:

1. Updated project folder
2. Database file
3. Screenshots:
 - Registration
 - Login
 - Dashboard
4. README explaining authentication flow

12. Learning Outcome

After Task-2, interns will:

- Understand authentication systems
- Be able to build secure web apps
- Be ready for APIs, roles, and deployment

13. YouTube Learning Support (Maximum Help)

Flask Authentication

- Flask Login System Tutorial
<https://www.youtube.com/watch?v=71EU8gnZqZQ>
- Flask Authentication Explained
<https://www.youtube.com/watch?v=dam0GPOAvVI>

Password Security

- Password Hashing Explained
<https://www.youtube.com/watch?v=8ZtlnCIXeIQ>

14. Free Documentation & Study Material

- Flask Sessions
<https://flask.palletsprojects.com/en/latest/quickstart/#sessions>
- Werkzeug Security
<https://werkzeug.palletsprojects.com>
- OWASP Password Guidelines
<https://owasp.org/www-project-top-ten/>